

Contiguous Storage

Objectives

How do you manage group data efficiently?

- ◆ Store
- ◆ Input
- ◆ Output
- ◆ Search
- ◆ Sort

Contents

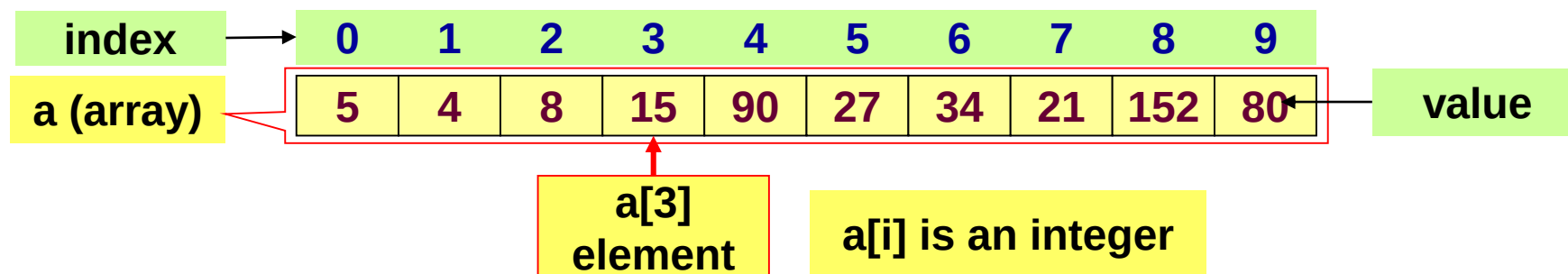
- ◆ Introduction to contiguous storage
- ◆ Arrays
- ◆ One-dimensional Arrays
 - Declaration
 - Memory Allocation
 - Initialization
 - Accessing elements
 - Traversing
 - 1-D Arrays are parameters of functions
 - Searching
 - Sorting
- ◆ 2-D Arrays
- ◆ User-defined Data Type: Structure

1- Contiguous Storage

- ◆ Commonly, a set of the same meaning elements are considered.
- ◆ They are stored in a contiguous block of memory.
- ◆ Ex: Group of 10 int numbers → 40 bytes block is needed.
- ◆ Data are considered can be a group of some items which belong to some different data types → Contiguous memory block is partitioned into some parts which have different size, one part for an item.
- ◆ Data structure: A structure of data stored.
- ◆ Array is the simplest data structure which contains some items which belong to the same data type.
- ◆ Common used operations on a group: Add, Search, Remove, Update, Sort

2 - Array

- An **array** is a data structure consisting of an ordered set of elements of common type that are stored contiguously in memory. Each element is identified by its position (index).



- Dimension:** Direction that is used to perform an action on array.
- Number of dimensions:** Number of indexes are used to specify an element.
- Common arrays:** 1-D and 2-D arrays.
- Name of an array:** An array has its name.

column

m

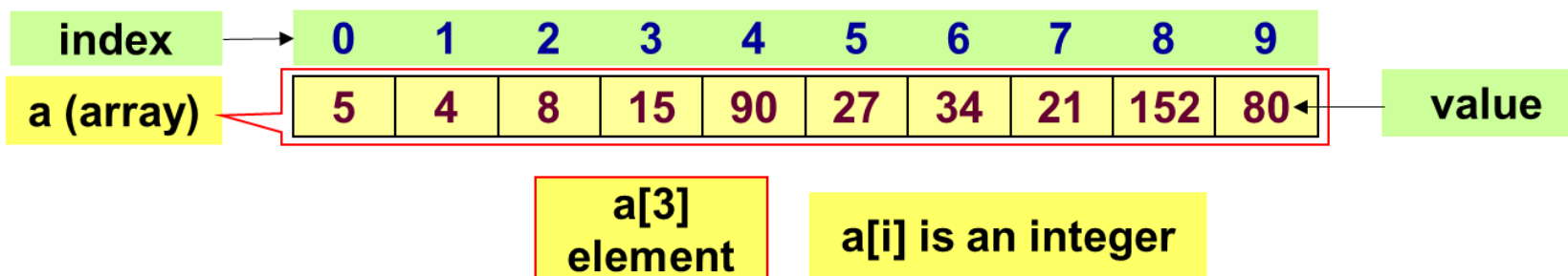
row

	0	1	2	3	4
0	1	7	6	3	7
1	2	-9	2	5	8
2	-5	40	0	5	9

m[1][3]

3 - One-dimensional Arrays (1-D)

- ◆ **1-D array:** a collection of items (elements, terms) which belong to the **same data type** and are **stored contiguously in memory**.
 - Each element has a unique index and holds a single value. Index numbering starts at 0 and extends to one less than the number of elements in the array.
 - To refer to a specific element, we write the array name followed by bracket notation around the element's index. Example: `identifier[index]`
- ◆ 1-D array structure:



1-D Array: Declaration

- ◆ If the array is stored in the stack segment → Use a **STATIC** array → The compiler will determine the array's storage at compile-time.

- ◆ Syntax:

```
DataType ArrayName[NumberOfElements];
```

- ◆ Example:

```
int a1[5]; char s[12]; double a2[100];
```

- ◆ How compilers can determine the memory size of an array?

=> NumberOfElements * sizeof(dataType) → int a1[5] → 5 * sizeof(int) = 5*4 = 20 bytes

1-D Array: Declaration (cont.)

- ◆ If the array is stored in the heap → Use a pointer (DYNAMIC array) → The array's storage will be allocated in the heap at run-time through memory allocating functions (malloc, calloc, realloc)
- ◆ Example:

```
#include <stdio.h>
#include <stdlib.h>

int main(){
    int *arr = (int *)calloc(5, sizeof(int)); // Allocates and initializes memory for 5 integers
    if (arr == NULL) {
        printf("Memory allocation failed\n");
    }

    system("pause");
    return 0;
}
```


1-D Array: Example Memory Allocation

```
#include <stdio.h>
#include <stdlib.h>
```

```
int MAX = 20;
```

```
int main()
```

```
{
```

```
    printf("MAX address = %u\n", &MAX);
```

```
    printf("main() address = %u\n", &main);
```

```
    int a1[5]; /* Static array of 5 integer numbers */
```

```
    double *a2 = NULL; /* Dynamic array of double numbers */
```

```
    /* Allowcate a memory block for 10 double numbers */
```

```
    a2 = (double*)calloc(10, sizeof(double));
```

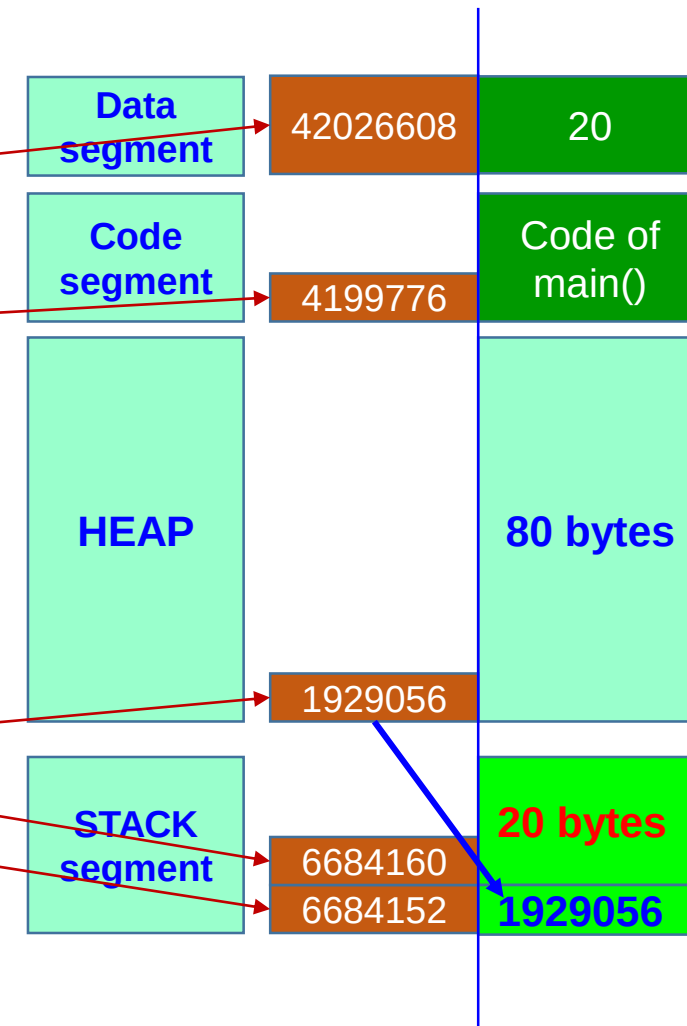
```
    printf("a1 address = %u\n", &a1);
```

```
    printf("a2 address = %u\nvalue of a2 = %u\n", &a2, a2);
```

```
    system("pause");
```

```
    return 0;
```

```
}
```



```
MAX address = 4206608
main() address = 4199776
a1 address = 6684160
a2 address = 6684152
value of a2 = 1929056
```

1-D Arrays: Initialization & Accessing Elements

- ◆ Initialize an array:

```
DataType a[] = {value1, value2, ... };
```

- ◆ How to access the i^{th} element of the array **a**?

a is the address of the first element. Based on operation on pointers:

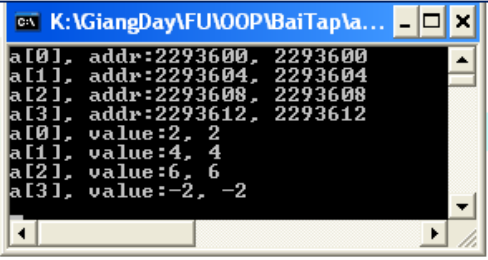
→ **$a+i$** : address of the i^{th} element, another way: **$\&a[i]$**

→ **$*(a+i)$** : value of the i^{th} element, another way: **$a[i]$**

1-D Arrays: Initialization & Accessing Elements (cont.)

Compiler will automatically count number of initial values to determine the size of array memory

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int a[] = {2, 4, 6, -2};
    int i;
    for (i=0; i<4; i++)
        printf("a[%d], addr:%u, %u\n", i, a+i, &a[i]);
    for (i=0; i<4; i++)
        printf("a[%d], value:%d, %d\n", i, *(a+i), a[i]);
    getchar();
    return 0;
}
```

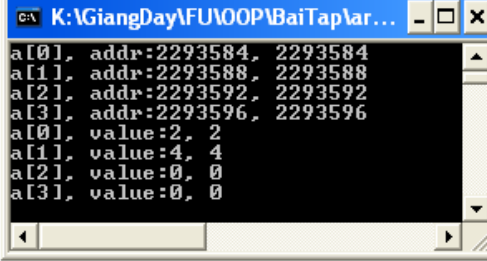


Index	Address	Value
a[0]	2293600	2
a[1]	2293604	4
a[2]	2293608	6
a[3]	2293612	-2

The size of array memory is pre-defined.
Compiler will fill 0 to elements which are not initialized.

`int a[5];`
Elements contain un-predictable values because they are local variables.
TEST IT !!!!

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int a[5] = {2, 4};
    int i;
    for (i=0; i<4; i++)
        printf("a[%d], addr:%u, %u\n", i, a+i, &a[i]);
    for (i=0; i<4; i++)
        printf("a[%d], value:%d, %d\n", i, *(a+i), a[i]);
    getchar();
    return 0;
}
```



Index	Address	Value
a[0]	2293584	2
a[1]	2293588	4
a[2]	2293592	0
a[3]	2293596	0
a[4]	2293600	0

1-D Arrays: Traversing

- ◆ A way to visit each element of an array
- ◆ Suppose that the 1-D array, named **a**, containing **n** elements.
- ◆ **Forward traversal:**

```
int i;  
for (i=0; i<n; i++){  
    [if (condition)] Access a[i];  
}
```

- ◆ **Backward traversal:**

```
int i;  
for (i=n-1; i>=0; i--){  
    [if (condition)] Access a[i];  
}
```

1-D Array is a Function Parameter

The array parameter of a function is the pointer of the first element of the array.

◆ Example 1:

- Input an array of n integers → **void input (int* a, int n)**

◆ Example 2:

- Input elements of an array of integers which its number of element is stored at the pointer **pn** → **void input (int a[], int*pn)**

◆ Example 3:

- Calculate the sum of an array of n integers → **int sum (int *a, int n)**

◆ Example 4:

- Output an array of n double numbers → **void output (double a[], int n)**

1-D Array is a Function Parameter: Demo

◆ Demo 1: Develop a C-program that will:

- Accept values to an integer array that may contain 100 elements.
- Print out the it's maximum value.
- Print out it's elements.
- Print out it's even values.

◆ *Hint:* • **Nouns:**

- Constant: `MAXN=100`
- Static array of integers → `int a[MAXN]`
- Real number of elements → `int n`
- Maximum value → `int maxVal`.

• **Verbs:**

- Begin
- Input n (one value)
- Input a, n (**function**)
- maxVal = get maximum value in a, n (**function**)
- Print out maxVal (one value)
- Print out a, n (**function**)
- Print even values in a, n (**function**)
- End

Array Function Parameter: Demo 1 (cont.)

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #define MAXN 100
4
5  /* Prototypes */
6  void input(int *a, int n);
7  int max(int a[], int n);
8  void print(int *a, int n);
9  void printEven(int *a, int n);
10
11 int main()
12 {
13     int a[MAXN]; // static array of 100 integers
14     int n; // real used number of elements
15     int maxVal;
16     do{
17         printf("How many elements which be used 1 ... %d: ", MAXN);
18         scanf("%d", &n);
19     }while(n<1 || n>MAXN);
20     printf("Enter %d values of the array:\n", n);
21     input(a, n);
22     maxVal = max(a, n);
23     printf("Max value: %d\n", maxVal);
24     printf("\nInputted array: ");
25     print(a, n);
26     printf("\nEven values in array: ");
27     printEven(a,n);
28     printf("\n");
29     while(getchar()!='\n'); // Clear buffer
30
31     system("pause");
32     return 0;
33 }
```

D:\MonHoc\PRF192\ThucHanh\array_demo.exe

How many elements which be used 1 ... 100: 6

Enter 6 values of the array:

3 5 8 1 2 0

Max value: 8

Inputted array: 3 5 8 1 2 0

Even values in array: 8 2 0

Press any key to continue . . .

```
36 void input(int *a, int n){
37     /* Use forward traversal, accept each value*/
38     int i;
39     for(i=0; i<n; i++){
40         scanf("%d", &a[i]);
41     }
42 }
43
44 int max(int a[], int n){
45     int result = a[0];
46     /* Use forward traversal, compare each value with result*/
47     int i;
48     for(i=0; i<n; i++){
49         if(result<a[i]){
50             result = a[i];
51         }
52     }
53     return result;
54 }
```

```
56 void print(int *a, int n){
57     /* Use forward traversal, print out each value*/
58     int i;
59     for(i=0; i<n; i++){
60         printf("%d ", a[i]);
61     }
62 }
63
64 void printEven(int *a, int n){
65     /* Use forward traversal, print out each even value*/
66     int i;
67     for(i=0; i<n; i++){
68         if(a[i]%2==0){
69             printf("%d ", a[i]);
70         }
71     }
72 }
```


Array Function Parameter: Demo 1

◆ Problems:

- If you allocate an array having 100 elements but 6 elements are used then memory is wasted.
- If you allocate an array having 100 elements but 101 elements are used then there is a lack of memory.

◆ Solution:

- Use a dynamic array
- Can expand the size of the original array

Array Function Parameter: Solution

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #define MAXN 100
4
5  /* Prototypes */
6  void input(int *a, int n);
7  int max(const int *a, int n);
8  void print(const int *a, int n);
9  void printEven(const int *a, int n);
10
11 int main() {
12     int *a; // dynamic array
13     int n;  // real used number of elements
14     int maxVal;
15
16     do {
17         printf("How many elements will be used (1 ... %d): ", MAXN);
18         scanf("%d", &n);
19     } while (n < 1 || n > MAXN);
20
21     a = (int *)calloc(n, sizeof(int));
22     if (a == NULL) {
23         printf("Memory allocation failed!\n");
24         return 1;
25     }
26
27     printf("Enter %d values of the array:\n", n);
28     input(a, n);
```

Use Dynamic Array

Solution (cont.)

```
30 maxVal = max(a, n);
31 printf("\nMax value: %d\n", maxVal);
32
33 printf("\nInputted array: ");
34 print(a, n);
35
36 printf("\nEven values in array: ");
37 printEven(a, n);
38
39 // Allow user to resize array
40 int newSize;
41 printf("\nEnter new size for the array (greater than %d): ", n);
42 scanf("%d", &newSize);
43
44 if (newSize > n) {
45     a = (int *)realloc(a, newSize * sizeof(int));
46     if (a == NULL) {
47         printf("Reallocation failed!\n");
48         free(a);
49         return 1;
50     }
```

Expand the size of
the original array

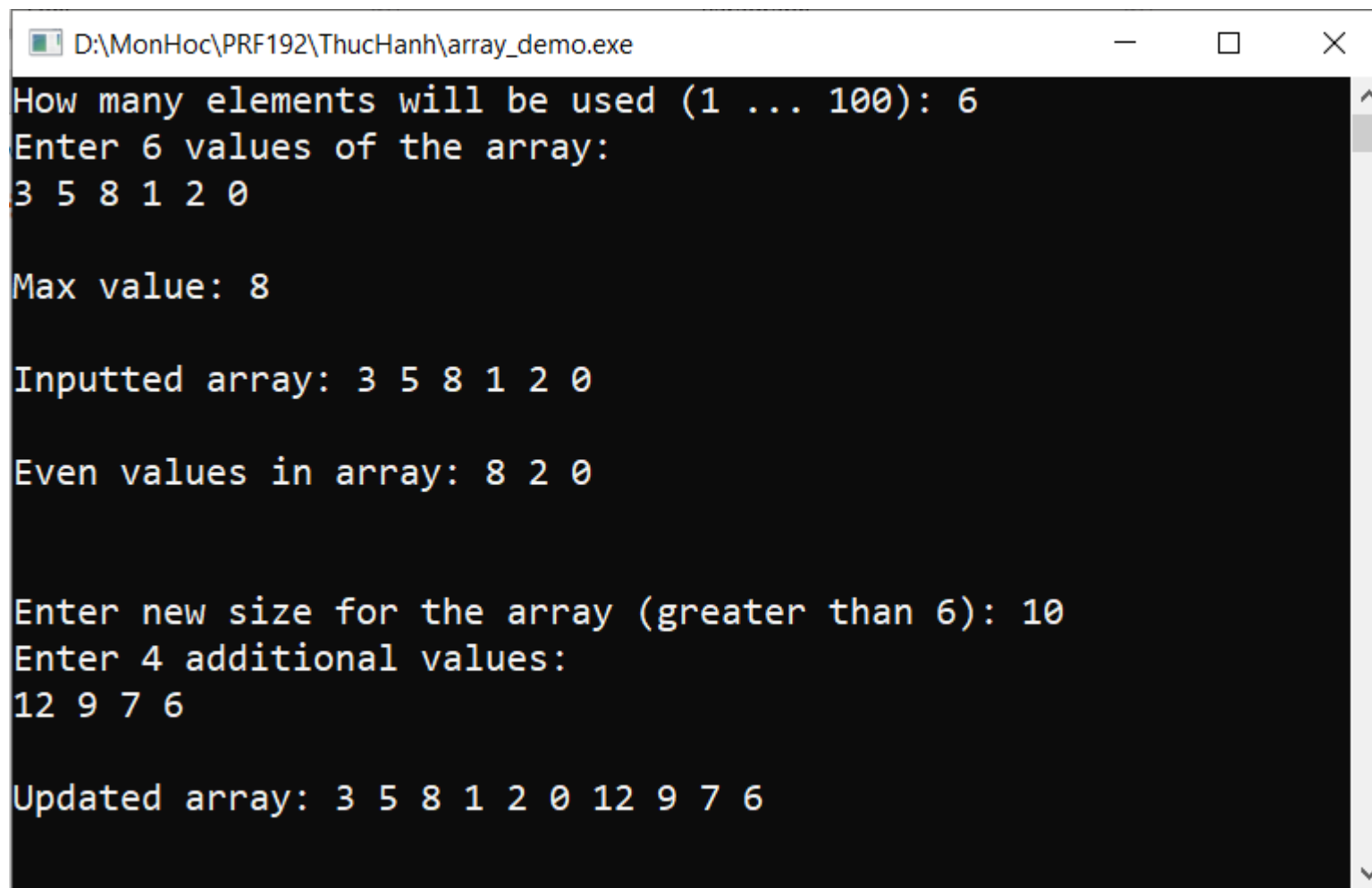
```
51     printf("Enter %d additional values:\n", newSize - n);
52     input(a + n, newSize - n);
53     n = newSize;
54
55     printf("\nUpdated array: ");
56     print(a, n);
57 } else {
58     printf("New size must be greater than the current size (%d).\n", n);
59 }
60
61 free(a); // Free allocated memory
62 return 0;
63 }
```

Solution (cont.)

```
65  /* Function definitions */
66
67  void input(int *a, int n) {
68      for (int i = 0; i < n; i++) {
69          scanf("%d", a + i); // Use pointer arithmetic
70      }
71  }
72
73  int max(const int *a, int n) {
74      int result = *a; // Dereference pointer to get the first value
75      for (int i = 1; i < n; i++) {
76          if (result < *(a + i)) { // Use pointer arithmetic
77              result = *(a + i);
78          }
79      }
80      return result;
81  }
```

```
83  void print(const int *a, int n) {
84      for (int i = 0; i < n; i++) {
85          printf("%d ", *(a + i)); // Use pointer arithmetic
86      }
87      printf("\n");
88  }
89
90  void printEven(const int *a, int n) {
91      for (int i = 0; i < n; i++) {
92          if (*(a + i) % 2 == 0) { // Use pointer arithmetic
93              printf("%d ", *(a + i));
94          }
95      }
96      printf("\n");
97  }
```

Output Solution



```
D:\MonHoc\PRF192\ThucHanh\array_demo.exe

How many elements will be used (1 ... 100): 6
Enter 6 values of the array:
3 5 8 1 2 0

Max value: 8

Inputted array: 3 5 8 1 2 0

Even values in array: 8 2 0

Enter new size for the array (greater than 6): 10
Enter 4 additional values:
12 9 7 6

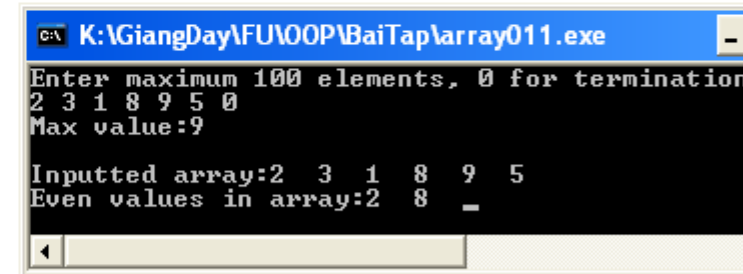
Updated array: 3 5 8 1 2 0 12 9 7 6
```

Exercise 1:

- ◆ Develop a C-program that will:
 - Accept values to an integer array that may contains 100 elements. The input will terminate when user enters the value of zero.
 - Print out the it's maximum value.
 - Print out it's elements.
 - Print out it's even values.
- ◆ *Requirement:*
 - The difference between this problem with the previous one is the input operation can terminate abruptly when 0 is accepted.
 - Memory block of the array needs to be allocated in excess
 - The function for input values of the array must be modified for this case and the number of elements is updated after each valid value is accepted.

Exercise 1 (cont.)

```
2 #include <stdio.h>
3 #define MAXN 100
4 /* Input an array, number of elements is stored at pn
5    User will terminate inputting when 0 is entered.*/
6 void input(int*a, int *pn);
7 int max(int a[], int n);
8 void print (int* a, int n);
9 void printEven (int* a, int n);
10 int main()
11 {   int a[MAXN]; /* static array of 100 integers */
12     int n; /* real used number of elements */
13     int maxVal;
14     input(a, &n);
15     maxVal = max (a, n);
16     printf("Max value:%d\n", maxVal);
17     printf("\nInputted array:");
18     print(a, n);
19     printf("\nEven values in array:");
20     printEven(a, n);
21     while (getchar() != '\n'); getchar();
22     return 0;
23 }
```



```
K:\GiangDay\FU\OOP\BaiTap\array011.exe
Enter maximum 100 elements, 0 for termination
2 3 1 8 9 5 0
Max value:9

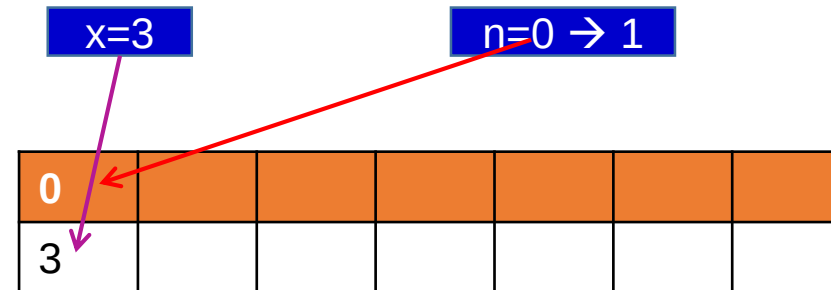
Inputted array:2 3 1 8 9 5
Even values in array:2 8
```

Exercise 1 (cont.)

```

24 void input(int*a, int *pn)
25 { *pn=0; /* reset the number of elements */
26   printf ("Enter maximum %d elements, 0 for termination\n", MAXN);
27   int x; /* inputted value */
28   do
29   { scanf("%d", &x);
30     if (x!=0) a[( *pn)++] = x;
31   }
32   while (x!=0 && *pn < MAXN);
33 }

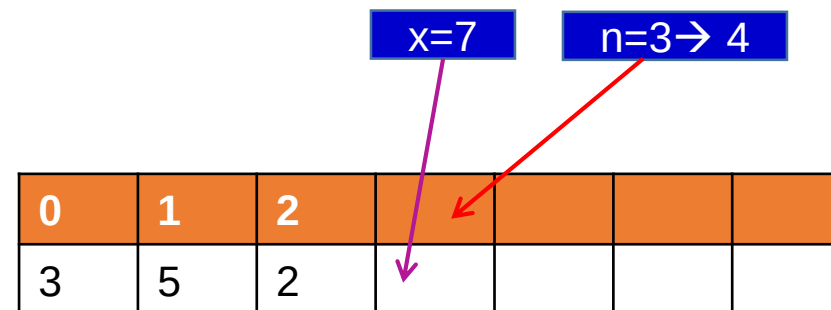
```



```

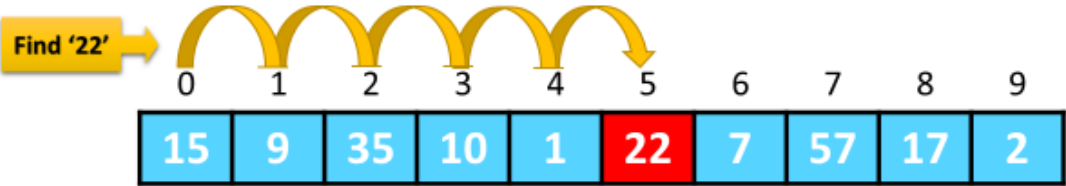
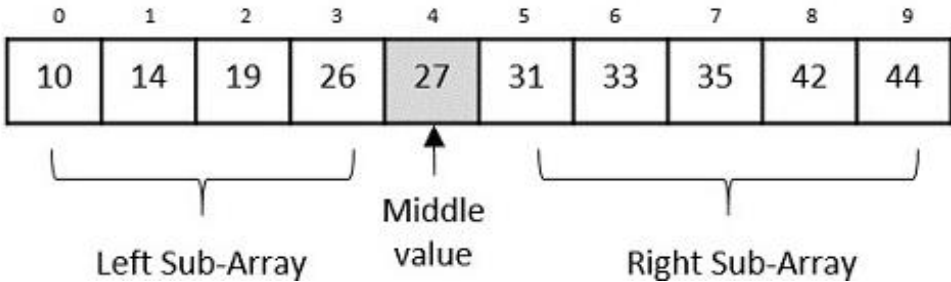
34 int max(int a[], int n)
35 {
36   /* Do yourself */
42 }
43 void print (int* a, int n)
44 { /* Do yourself */
47 }
48 void printEven (int* a, int n)
49 { /* Do yourself */
53 }

```



1-D Arrays: Searching

- ◆ A search algorithm finds the record of interest using the key array.
- ◆ Return value: The positional index at which the interest value is found.
- ◆ Two common search algorithms are:

Linear search	Binary search
 The diagram illustrates linear search on an array of 10 elements. A yellow box labeled 'Find '22'' has an arrow pointing to the first element at index 0. Yellow curved arrows show the search process moving sequentially from index 0 to index 5. The element at index 5, which is 22, is highlighted in red. The array values are: 15, 9, 35, 10, 1, 22, 7, 57, 17, 2.	 The diagram illustrates binary search on a sorted array of 10 elements. The array values are: 10, 14, 19, 26, 27, 31, 33, 35, 42, 44. The middle element at index 4 (value 27) is highlighted in grey and labeled 'Middle value' with an upward arrow. Brackets below the array divide it into a 'Left Sub-Array' (indices 0-3) and a 'Right Sub-Array' (indices 5-9).

Searching: Linear Search

- ◆ **Linear Search:** Find the position of the value x in the array a having n elements.

Search the value of 6 in the array a having 8 items.

5	9	2	7	6	5	2	5
i=0	1	2	3	4			

Search the value of 12 in the array a having 8 items.

5	9	2	7	6	5	2	5
i=0	1	2	3	4	5	6	7

-1

```
int firstLinearSearch(int x, int a[], int n)
{
    int i;
    for ( i=0; i<n; i++)
        if ( x == a[i] ) return i;
    return -1;
}
```

```
int lastLinearSearch(double x, double *a, int n)
{
    int i;
    for ( i=n-1; i>=0; i--)
        if ( x == a[i] ) return i;
    return -1;
}
```

Exercise 2: Using Linear Search algorithm

```
#include <stdio.h>
int firstLinearSearch ( int x, int a[], int n)
{
    /* Your code */
}
int lastLinearSearch ( int x, int a[], int n)
{
    /* Your code */
}
int main()
{
    int a[] = { 3,34,5,1,2,8,9,2,9 }, x=2;
    int pos1= firstLinearSearch(x,a,9);
    if (pos1>=0)
    {
        int pos2= lastLinearSearch(x,a,9);
        printf("First existence:%d, last existence:%d\n", pos1, pos2);
    }
    else printf("%d does not exist!\n", x);
    getchar();
    return 0;
}
```

Do yourself



The screenshot shows a command prompt window titled "K:\GiangDay\FU\OOP\BaiTap\array02.exe". The output displayed is "First existence:4, last existence:7".

Searching: Binary Search

- Binary Search: Condition for application: Values in the array were sorted.

```
int binarySearch ( int x, int a[], int n)
{
    int i=0, j= n-1, c ;
    while (i<=j)
    {
        c= (i+j)/2;
        if ( x== a[c] ) return c ;
        if ( x < a[c] ) j = c-1;
        else i = c +1;
    }
    return -1;
}
```

	$i \xrightarrow{\hspace{1.5cm}}$										j
	0	1	2	3	4	5	6	7	8	9	10
a	4	6	7	8	10	12	13	15	17	18	19

x= 15
c=(i+j)/2=5 15 elements are considered

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

x= 15
c=(i+j)/2=8 5 elements are considered

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

x= 15
c=(i+j)/2=6 2 elements considered

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

x= 15
c=(i+j)/2=7 1 element considered

return c (7)

	i → j										
	0	1	2	3	4	5	6	7	8	9	10
a	4	6	7	8	10	12	13	15	17	18	19

x=16
c=(i+j)/2=5 15 elements are considered

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

x=16
c=(i+j)/2=8 5 elements are considered

																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																		</
--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	----

x=16
c=(i+j)/2=6 2 elements considered

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

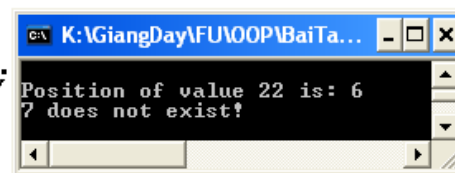
x=16
c=(i+j)/2=7 1 element considered

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

i>j → return -1

Exercise 3: Using Binary Search algorithm

```
#include <stdio.h>
int binarySearch (int x, int a[], int n)
{
    /* YOUR CODE */
}
int main()
{
    int a[] = { 1, 4, 8, 10, 12, 16, 22, 24 };
    int n=8, k1= 22, k2= 7;
    int pos1= binarySearch(k1,a,n);
    int pos2= binarySearch(k2,a,n);
    if (pos1>=0) printf("\nPosition of value %d is: %d", k1, pos1);
    else printf("\n%d does not exist!", k1);
    if (pos2>=0) printf("\nPosition of value %d is: %d", k2, pos2);
    else printf("\n%d does not exist!", k2);
    getchar();
    return 0;
}
```

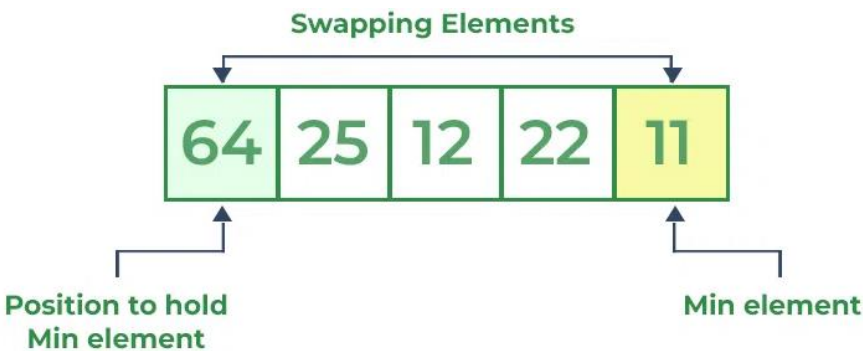
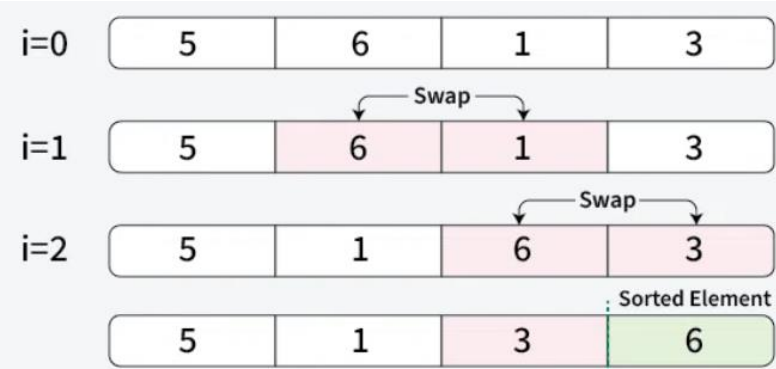


Evaluation:

No. of elements considered	No. of comparisons
$n = 2^m$	1
2^{m-1}	1
2^{m-2}	1
...	...
2^0	1
Sum	$m+1 = \log_2(n) + 1$

1-D Arrays: Sorting

- ◆ **Sorting:** Changing positions of elements in an array so that values are in a order based on a pre-defined order relation.
- ◆ Default order relation in set of numbers: Value order
- ◆ Default order relation in a set of characters/ strings: Dictionary order
- ◆ Only two sorting algorithms are introduced here.

Selection sort	Bubble sort
 The diagram shows an array with five elements: 64, 25, 12, 22, and 11. The element 11 is highlighted in yellow and labeled 'Min element'. An arrow points from the position of 64 to the position of 11, labeled 'Swapping Elements'. Another arrow points to the position of 64, labeled 'Position to hold Min element'.	 The diagram shows four rows of an array with four elements each. Row 1 (i=0): 5, 6, 1, 3. Row 2 (i=1): 5, 6, 1, 3. Row 3 (i=2): 5, 1, 6, 3. Row 4: 5, 1, 3, 6. Arrows labeled 'Swap' indicate swaps between 6 and 1 in row 1, and between 6 and 3 in row 2. The final row shows the array 5, 1, 3, 6, with the element 6 highlighted in green and labeled 'Sorted Element'.

Sorting: Selection Sort

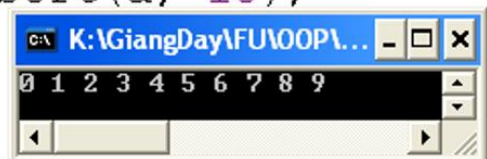
- Find the minimum value in the list
- Swap it with the value in the first position
- Repeat the steps above for remainder of the list

	0	1	2	3	4	5	6	Lần		Số lần so sánh
a	4	2	6	9	3	5	1	i=0	minIndex=i; for (j=i+1 ; j<n; j++) if (a[minIndex]>a[j]) minIndex=j; Swap(a[i],a[minIndex]);	6
a	1	2	6	9	3	5	4	i=1	minIndex=i; for (j=i+1 ; j<n; j++) if (a[minIndex]>a[j]) minIndex=j; Swap(a[i],a[minIndex]);	5
a	1	2	6	9	3	5	4	i=2	minIndex=i; for (j=i+1 ; j<n; j++) if (a[minIndex]>a[j]) minIndex=j; Swap(a[i],a[minIndex]);	4
a	1	2	3	9	6	5	4	i=3	minIndex=i; for (j=i+1 ; j<n; j++) if (a[minIndex]>a[j]) minIndex=j; Swap(a[i],a[minIndex]);	3
a	1	2	3	4	6	5	9	i=4	minIndex=i; for (j=i+1 ; j<n; j++) if (a[minIndex]>a[j]) minIndex=j; Swap(a[i],a[minIndex]);	2
a	1	2	3	4	5	6	9	i=5	minIndex=i; for (j=i+1 ; j<n; j++) if (a[minIndex]>a[j]) minIndex=j; Swap(a[i],a[minIndex]);	1
a	1	2	3	4	5	6	9			
Xong										
Với n=7, Số lần so sánh = 6+5+4+3+2+1 = 7 (6)/2										
Tổng quát: Số lần so sánh: n(n-1)/2										

Selection Sort: Students complete the demo

```
2 #include <stdio.h>
3 void ascSelectionSort( int* a, int n)
4 { int minIndex; /* index of min. value in a group */
5   int i,j ; /* vars for looping */
6   /* Group begins at position i to n-1*/
7   for (i=0; i< n-1; i++)
8   { minIndex = i; /* init minimum position */
9     /* update minIndex of the group at i, i+1,..., n-1*/
10    for (j=i+1; j<n; j++) if (a[minIndex]> a[j]) minIndex= j;
11    /* Move minimum value to the begin of the group */
12    if (minIndex > i)
13    { int t = a[minIndex];
14      a[minIndex] = a[i];
15      a[i] = t;
16    }
17  }
18 }
```

```
20 void print (int*a, int n)
21 { int i;
22   for (i=0; i<n; i++) printf("%d ", a[i]);
23 }
24 int main()
25 { int a[] = { 1,3,5,7,9,2,4,6,8, 0 };
26   ascSelectionSort(a, 10);
27   print(a,10);
28   getchar();
29   return 0;
30 }
```



Sorting: Bubble Sort

- ◆ It works by repeatedly stepping through the list to be sorted, comparing two items at a time and swapping them if they are in the wrong order.
- ◆ The pass through the list is repeated until no swaps are needed, which means the list is sorted

$X = \{7, 2, 5, 4, 1, 3, 8, 6\}$

Dãy ban đầu: 7, 2, 5, 4, 1, 3, 8, 6

$i=0, j=7$, swap($X[j-1], X[j]$), ta có 7, 2, 5, 4, 1, 3, 6, 8

$i=0, j=6$, không đổi chỗ, ta có 7, 2, 5, 4, 1, 3, 6, 8

$i=0, j=5$, không đổi chỗ, ta có 7, 2, 5, 4, 1, 3, 6, 8

$i=0, j=4$, swap($X[j-1], X[j]$), ta có 7, 2, 5, 1, 4, 3, 6, 8

$i=0, j=3$, swap($X[j-1], X[j]$), ta có 7, 2, 1, 5, 4, 3, 6, 8

$i=0, j=2$, swap($X[j-1], X[j]$), ta có 7, 1, 2, 5, 4, 3, 6, 8

$i=0, j=1$, swap($X[j-1], X[j]$), ta có 1, 7, 2, 5, 4, 3, 6, 8

$i=1$, thực hiện tương tự ta có 1, 2, 7, 3, 5, 4, 6, 8

$i=2$, thực hiện tương tự ta có 1, 2, 3, 7, 4, 5, 6, 8

$i=3$, thực hiện tương tự ta có 1, 2, 3, 4, 7, 5, 6, 8

$i=4$, thực hiện tương tự ta có 1, 2, 3, 4, 5, 7, 6, 8

$i=5$, thực hiện tương tự ta có 1, 2, 3, 4, 5, 6, 7, 8

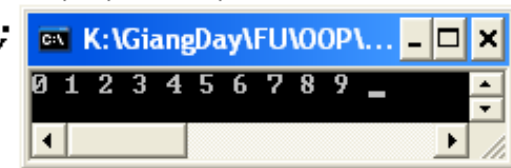
$i=6$, thực hiện tương tự ta có 1, 2, 3, 4, 5, 6, 7, 8

Cuối cùng ta có 1, 2, 3, 4, 5, 6, 7, 8, dãy đã được sắp xếp.

Bubble Sort: Demo

```
2 #include <stdio.h>
3 void ascBubbleSort( int* a, int n)
4 { int i,j ; /* vars for looping */
5   /* Loop n-1 pass */
6   for (i=0; i< n-1; i++)
7   { /* Go to the end of array to move the min value up */
8     for (j=n-1; j>i; j--)
9       /*The later element is smaller than the previous one*/
10      if (a[j]<a[j-1])
11      { /* move the smaller up */
12        int t = a[j];
13        a[j] = a[j-1];
14        a[j-1] = t;
15      }
16   }
17 }
```

```
18 void print (int*a, int n)
19 { int i;
20   for (i=0; i<n; i++)printf("%d ", a[i]);
21 }
22 int main()
23 { int a[] = { 1,3,5,7,9,2,4,6,8, 0 };
24   ascBubbleSort(a, 10);
25   print(a,10);
26   getchar();
27   return 0;
28 }
```



1-D Arrays: A Case Study

- ◆ Develop a C-program that helps user managing an 1-D array of integers (maximum of 100 elements) using the following simple menu:
 - 1- Add a value
 - 2- Search a value
 - 3- Remove the first existence of a value
 - 4- Remove all existences of a value
 - 5- Print out the array
 - 6- Print out the array in ascending order (positions of elements are preserved)
 - 7- Print out the array in descending order (positions of elements are preserved)
 - Others- Quit

Case Study: Problem Analyze

- ◆ In this program, user can freely add or remove one or more elements to/ from the array. So, an extra memory allocation is needed (100 items).

- ◆ **Data:**

Array of integers `int a[100], n`

Searched/added/removed number → int value

Case Study: Problem Analyze (cont.)

◆ Functions (cont.):

- `int menu()` → Get user choice
- `int isFull(int *a, int n)` - Testing whether an array is full or not
- `int isEmpty(int *a, int n)` - Testing whether an array is empty or not
- `void add(int x, int*a, int*pn)` → adding an element to the array will increase number of elements
- `int search(int x, int *a, int n)` → return a position found in the array
- `int removeOne (int pos, int*a, int*pn)` → Removing a value at the position pos will decrease number of elements → return 1: successfully, 0: fail
- `int remove All(int x, int*a, int*pn)` → Removing a value will decrease number of elements → return 1: successfully, 0: fail
- `void printAsc(int*a, int n)` – printing array, elements are preserved
- `void printDesc(int*a, int n)` – printing array, elements are preserved
- `void print(int*a, int n)`

Case Study: Code Design

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  #define MAX_SIZE 100
5
6  /* Function Prototypes */
7  void addValue(int *a, int *n, int value);
8  int searchValue(int *a, int n, int value);
9  void removeFirst(int *a, int *n, int value);
10 void removeAll(int *a, int *n, int value);
11 void printArray(int *a, int n);
12 void printAscending(int *a, int n);
13 void printDescending(int *a, int n);
14
15 int main() {
16     int a[MAX_SIZE]; // Array to store integers
17     int n = 0;        // Number of elements in the array
18     int choice, value;
19
20     do {
21         printf("\nMenu:\n");
22         printf("1- Add a value\n");
23         printf("2- Search a value\n");
24         printf("3- Remove the first existence of a value\n");
25         printf("4- Remove all existences of a value\n");
26         printf("5- Print out the array\n");
27         printf("6- Print out the array in ascending order (positions of elements are preserved)\n");
28         printf("7- Print out the array in descending order (positions of elements are preserved)\n");
29         printf("Others- Quit\n");
30         printf("Your choice: ");
31         scanf("%d", &choice);
```

Case Study: Code Design

```
33 ☐
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54

switch (choice) {
    case 1:
        printf("Enter value to add: ");
        scanf("%d", &value);
        addValue(a, &n, value);
        break;

    case 2:
        printf("Enter value to search: ");
        scanf("%d", &value);
        int pos = searchValue(a, n, value);
        if (pos != -1)
            printf("Value %d found at position %d.\n", value, pos);
        else
            printf("Value %d not found.\n", value);
        break;

    case 3:
        printf("Enter value to remove (first occurrence): ");
        scanf("%d", &value);
        removeFirst(a, &n, value);
        break;
```

Case Study: Code Design

```
56     case 4:
57         printf("Enter value to remove (all occurrences): ");
58         scanf("%d", &value);
59         removeAll(a, &n, value);
60         break;
61
62     case 5:
63         printf("Current array: ");
64         printArray(a, n);
65         break;
66
67     case 6:
68         printf("Array in ascending order: ");
69         printAscending(a, n);
70         break;
71
72     case 7:
73         printf("Array in descending order: ");
74         printDescending(a, n);
75         break;
76
77     default:
78         printf("Goodbye!\n");
79     }
80 } while (choice >= 1 && choice <= 7);
81
82 return 0;
83 }
```


Case Study: Code Design

```
85  /* Add a value to the array */
86  void addValue(int *a, int *n, int value) {
87      if (*n >= MAX_SIZE) {
88          printf("Array is full. Cannot add more values.\n");
89          return;
90      }
91      a[*n] = value;
92      (*n)++;
93      printf("Value %d added successfully.\n", value);
94  }
95
96  /* Search for a value in the array */
97  int searchValue(int *a, int n, int value) {
98      for (int i = 0; i < n; i++) {
99          if (a[i] == value) return i;
100     }
101     return -1;
102 }
```

```
104 /* Remove the first occurrence of a value */
105 void removeFirst(int *a, int *n, int value) {
106     int pos = searchValue(a, *n, value);
107     if (pos == -1) {
108         printf("Value %d not found. No removal performed.\n", value);
109         return;
110     }
111     for (int i = pos; i < *n - 1; i++) {
112         a[i] = a[i + 1];
113     }
114     (*n)--;
115     printf("Value %d removed successfully (first occurrence).\n", value);
116 }
```

Case Study: Code Design

```
118  /* Remove all occurrences of a value */
119  void removeAll(int *a, int *n, int value) {
120      int count = 0;
121      for (int i = 0; i < *n; ) {
122          if (a[i] == value) {
123              for (int j = i; j < *n - 1; j++) {
124                  a[j] = a[j + 1];
125              }
126              (*n)--;
127              count++;
128          } else {
129              i++;
130          }
131      }
132      if (count > 0)
133          printf("Value %d removed %d time(s).\n", value, count);
134      else
135          printf("Value %d not found. No removal performed.\n", value);
136  }
137
138  /* Print the array */
139  void printArray(int *a, int n) {
140      for (int i = 0; i < n; i++) {
141          printf("%d ", a[i]);
142      }
143      printf("\n");
144  }
```

Case Study: Code Design

```
16 void PrintASC(int *a,int n){
17
18     int *b=(int*)calloc(n,sizeof(int));
19
20     for(int i=0;i<n;i++)
21         b[i]=a[i];
22
23     int temp;
24     for(int i=0;i<n-1;i++)
25         for(int j=n-1;j>i;j--)
26             if(b[j]<b[j-1]){
27                 temp=b[j];
28                 b[j]=b[j-1];
29                 b[j-1]=temp;
30             }
31
32
33     for(int i=0;i<n;i++)
34         printf("%d\t",b[i]);
35
36     free(b);
37
38 }
39
```

Case Study: Code Design

```
41 void PrintDesc(int *a,int n){
42
43     int *b=(int*)calloc(n,sizeof(int));
44
45     for(int i=0;i<n;i++)
46         b[i]=a[i];
47
48     int temp;
49     for(int i=0;i<n-1;i++)
50         for(int j=n-1;j>i;j--)
51             if(b[j]>b[j-1]){
52                 temp=b[j];
53                 b[j]=b[j-1];
54                 b[j-1]=temp;
55             }
56
57
58     for(int i=0;i<n;i++)
59         printf("%d\t",b[i]);
60
61     free(b);
62
63 }
64
65
66 void Print(int *a,int n){
67     for(int i=0;i<n;i++)
68         printf("%d\t",a[i]);
69 }
70
```

Compile & Run

Input: 9 integer numbers

```
Menu:
1- Add a value
2- Search a value
3- Remove the first existence of a value
4- Remove all existences of a value
5- Print out the array
6- Print out the array in ascending order (position)
7- Print out the array in descending order (position)
Others- Quit
Your choice: 1
Enter value to add: 0
Value 0 added successfully.
```

Print all elements

```
Menu:
1- Add a value
2- Search a value
3- Remove the first existence of a value
4- Remove all existences of a value
5- Print out the array
6- Print out the array in ascending order (position)
7- Print out the array in descending order (position)
Others- Quit
Your choice: 5
Current array: 0 2 8 9 7 3 2 4 2
```

Print the array in ASC order

```
Menu:
1- Add a value
2- Search a value
3- Remove the first existence of a value
4- Remove all existences of a value
5- Print out the array
6- Print out the array in ascending order (position)
7- Print out the array in descending order (position)
Others- Quit
Your choice: 6
Array in ascending order: 0 2 2 2 3 4 7 8 9
```

Remove the first exist of value

```
Menu:
1- Add a value
2- Search a value
3- Remove the first existence of a value
4- Remove all existences of a value
5- Print out the array
6- Print out the array in ascending order (position)
7- Print out the array in descending order (position)
Others- Quit
Your choice: 3
Enter value to remove (first occurrence): 8
Value 8 removed successfully (first occurrence).
```

Search a value

```
Menu:
1- Add a value
2- Search a value
3- Remove the first existence of a value
4- Remove all existences of a value
5- Print out the array
6- Print out the array in ascending order (position)
7- Print out the array in descending order (position)
Others- Quit
Your choice: 2
Enter value to search: 4
Value 4 found at position 7.
```

Print the array in DESC order

```
Menu:
1- Add a value
2- Search a value
3- Remove the first existence of a value
4- Remove all existences of a value
5- Print out the array
6- Print out the array in ascending order (position)
7- Print out the array in descending order (position)
Others- Quit
Your choice: 7
Array in descending order: 9 8 7 4 3 2 2 2 0
```

Exercise 4:

- ◆ Develop a C-program that helps user managing an 1-D array of real numbers (maximum of 100 elements) using the following simple menu:
 - 1- Add a value
 - 2- Search a value
 - 3- Print out the array
 - 4- Print out values in a range ($\text{minVal} \leq \text{value} \leq \text{maxVal}$, minVal and maxVal are inputted)
 - 5- Print out the array in ascending order (positions of elements are preserved)
 - Others- Quit

4 - Two-Dimensional Arrays

- ◆ A group of elements which belong the same data type and they are divided into some rows and some column (it is called as matrix also).
- ◆ Each element is identified by two indexes (index of row, index of column).

Traversing a matrix:

```
for ( i =0; i<row; i++)  
{  
    for ( j=0; j< column; j++)  
        [if (condition)] Access m[i][j];  
}
```

		column				
row	m	0	1	2	3	4
	0	1	7	6	3	7
	1	2	-9	2	5	8
	2	-5	40	0	5	9

Two-Dimensional Arrays: Example

```
1  #include <stdio.h>
2  /* Function Prototypes */
3  int calculateSum(int rows, int cols, int arr[rows][cols]);
4  void printMatrix(int rows, int cols, int arr[rows][cols]);
5
6  int main() {
7      int m, n;
8      printf("Enter the number of rows (m): ");
9      scanf("%d", &m);
10     printf("Enter the number of columns (n): ");
11     scanf("%d", &n);
12     int array[m][n];
13     printf("Enter the elements of the %dx%d array:\n", m, n);
14     for (int i = 0; i < m; i++) {
15         for (int j = 0; j < n; j++) {
16             printf("Element [%d][%d]: ", i + 1, j + 1);
17             scanf("%d", &array[i][j]);
18         }
19     }
20     printf("\nMatrix:\n");
21     printMatrix(m, n, array);
22     int sum = calculateSum(m, n, array);
23     printf("\nSum of all elements: %d\n", sum);
24     return 0;
25 }
```

D:\MonHoc\PRF192\ThucHanh\2d_dimension_array.exe

```
Enter the number of rows (m): 2
Enter the number of columns (n): 3
Enter the elements of the 2x3 array:
Element [1][1]: 1
Element [1][2]: 2
Element [1][3]: 7
Element [2][1]: 2
Element [2][2]: 8
Element [2][3]: 5

Matrix:
    1    2    7
    2    8    5

Sum of all elements: 25
```


Two-Dimensional Arrays: Example (cont.)

```
27  /* Function to calculate the sum of elements of the 2D array */
28  int calculateSum(int rows, int cols, int arr[rows][cols]) {
29      int sum = 0;
30      for (int i = 0; i < rows; i++) {
31          for (int j = 0; j < cols; j++) {
32              sum += arr[i][j];
33          }
34      }
35      return sum;
36  }
37
38  /* Function to print the 2D array as a matrix */
39  void printMatrix(int rows, int cols, int arr[rows][cols]) {
40      for (int i = 0; i < rows; i++) {
41          for (int j = 0; j < cols; j++) {
42              printf("%4d", arr[i][j]); // Format to align values in columns
43          }
44          printf("\n"); // New line after each row
45      }
46  }
```

Summary

- ◆ Array is the simplest data structure for a group of elements which belong to the same data type.
- ◆ Each element in an array is identified by one or more index beginning from 0.
- ◆ Number of dimensions: Number of indexes are used to identify an element.
- ◆ Static arrays → Stack segment

`DataType a[MAXN];`

`DataType m[MAXROW][MAXCOL];`

- ◆ Dynamic array: Use pointer and allocate memory using functions

`double *a = (double*)calloc(n, sizeof(double));`

`int** m = (int**) calloc(row, sizeof(int*));`

`for (i=0; i<row; i++) m[i]= (int*)calloc(col, sizeof(int));`

Summary

◆ Accessing elements in an array:

1-D Array (a)		2-D Array (m)	
Address	Value	Address	Value
&a[index]	a[index]	&m[i][j]	m[i][j]
a+index	*(a+index)		
Compiler determines the address of an element:			
$a + \text{index} * \text{sizeof}(\text{DataType})$		$m + (i * \text{NumCol} + j) * \text{sizeof}(\text{DataType})$	

◆ Common operations on arrays:

- Add an element
- Search an element
- Remove an element
- Input
- Output
- Sort

Summary

- ◆ Understanding what a Structure type is?
- ◆ When to use a Structure type in a program
- ◆ How to define a Structure type and use a Structure in a program
- ◆ Know how to work with the components of a Structure